

12.4 Comparing Sorting Algorithms

At this point, it might be useful for us to take a moment and consider all the algorithms we have studied in this book to sort an n -element sequence.

Considering Running Time and Other Factors

We have studied several methods, such as insertion-sort and selection-sort, that have $O(n^2)$ -time behavior in the average and worst case. We have also studied several methods with $O(n \log n)$ -time behavior, including heap-sort, merge-sort, and quick-sort. Finally, the bucket-sort and radix-sort methods run in linear time for certain types of keys. Certainly, the selection-sort algorithm is a poor choice in any application, since it runs in $O(n^2)$ time even in the best case. But, of the remaining sorting algorithms, which is the best?

As with many things in life, there is no clear “best” sorting algorithm from the remaining candidates. There are trade-offs involving efficiency, memory usage, and stability. The sorting algorithm best suited for a particular application depends on the properties of that application. In fact, the default sorting algorithm used by computing languages and systems has evolved greatly over time. We can offer some guidance and observations, therefore, based on the known properties of the “good” sorting algorithms.

Insertion-Sort

If implemented well, the running time of *insertion-sort* is $O(n + m)$, where m is the number of *inversions* (that is, the number of pairs of elements out of order). Thus, insertion-sort is an excellent algorithm for sorting small sequences (say, less than 50 elements), because insertion-sort is simple to program, and small sequences necessarily have few inversions. Also, insertion-sort is quite effective for sorting sequences that are already “almost” sorted. By “almost,” we mean that the number of inversions is small. But the $O(n^2)$ -time performance of insertion-sort makes it a poor choice outside of these special contexts.

Heap-Sort

Heap-sort, on the other hand, runs in $O(n \log n)$ time in the worst case, which is optimal for comparison-based sorting methods. Heap-sort can easily be made to execute in-place, and is a natural choice on small- and medium-sized sequences, when input data can fit into main memory. However, heap-sort tends to be outperformed by both quick-sort and merge-sort on larger sequences. A standard heap-sort does not provide a stable sort, because of the swapping of elements.

Quick-Sort

Although its $O(n^2)$ -time worst-case performance makes **quick-sort** susceptible in real-time applications where we must make guarantees on the time needed to complete a sorting operation, we expect its performance to be $O(n \log n)$ time, and experimental studies have shown that it outperforms both heap-sort and merge-sort on many tests. Quick-sort does not naturally provide a stable sort, due to the swapping of elements during the partitioning step.

For decades quick-sort was the default choice for a general-purpose, in-memory sorting algorithm. Quick-sort was included as the `qsort` sorting utility provided in C language libraries, and was the basis for sorting utilities on Unix operating systems for many years. It has long been the standard algorithm for sorting arrays of primitive type in Java. (We discuss sorting of object types below.)

Merge-Sort

Merge-sort runs in $O(n \log n)$ time in the worst case. It is quite difficult to make merge-sort run in-place for arrays, and without that optimization the extra overhead of allocate a temporary array, and copying between the arrays is less attractive than in-place implementations of heap-sort and quick-sort for sequences that can fit entirely in a computer's main memory. Even so, merge-sort is an excellent algorithm for situations where the input is stratified across various levels of the computer's memory hierarchy (e.g., cache, main memory, external memory). In these contexts, the way that merge-sort processes runs of data in long merge streams makes the best use of all the data brought as a block into a level of memory, thereby reducing the total number of memory transfers.

The GNU sorting utility (and most current versions of the Linux operating system) relies on a multiway merge-sort variant. **Tim-sort** (designed by Tim Peters) is a hybrid approach that is essentially a bottom-up merge-sort that takes advantage of initial runs in the data while using insertion-sort to build additional runs. Tim-sort has been the standard sorting algorithm in Python since 2003, and it has become the default algorithm for sorting arrays of object types, as of Java SE 7.

Bucket-Sort and Radix-Sort

Finally, if an application involves sorting entries with small integer keys, character strings, or d -tuples of keys from a discrete range, then **bucket-sort** or **radix-sort** is an excellent choice, for it runs in $O(d(n + N))$ time, where $[0, N - 1]$ is the range of integer keys (and $d = 1$ for bucket sort). Thus, if $d(n + N)$ is significantly “below” the $n \log n$ function, then this sorting method should run faster than even quick-sort, heap-sort, or merge-sort.